



PROGRAMAÇÃO WEB II · APRESENTAÇÃO DE PROJETO

Lista de Amigos

Um CRUD completo em PHP e MySQL, protegido por login com sessão

Projeto da Gabi · Profª Alice

O PROJETO

Uma agenda web de amigos

Cadastrar, listar, alterar e excluir amigos. E **só quem faz login** consegue ver os dados.

PHP

Toda a lógica roda no servidor: recebe formulários, fala com o banco e monta as páginas.

MySQL + mysqli

Banco pwii com as tabelas usuario e amigo, acessado pela extensão mysqli.

W3.CSS + Font Awesome

Visual responsivo com classes prontas e ícones nos botões.

USBWebserver

Servidor local com Apache e MySQL para rodar tudo em localhost.

ORGANIZAÇÃO

Cada arquivo tem uma função

Arquivo	Função	Tipo
index.php	Formulário de login	Página
loginAction.php	Confere nome e senha no banco e cria a sessão	Ação
principal.php	Menu com os botões Adicionar e Listar	Protegida
cadastro.php, listar.php...	Telas e ações do CRUD de amigos	Protegidas
conexaoBD.php	Dados de conexão com o banco, em um só lugar	Reutilizável
cabecalho.php / rodape.php	Início e fim do HTML de todas as páginas	Reutilizável
verificarAcesso.php	Barra quem não fez login	Segurança
acessoNegado.php	Aviso para quem tentou entrar sem login	Página
logoutAction.php	Encerra a sessão e volta ao login	Ação

Duas tabelas no banco pwii

usuario

Campo	Tipo
idusuario	INT · PK · AI
nome	VARCHAR(45)
senha	VARCHAR(45)

Guarda quem pode entrar. Usuário de teste: gabi / gabi123.

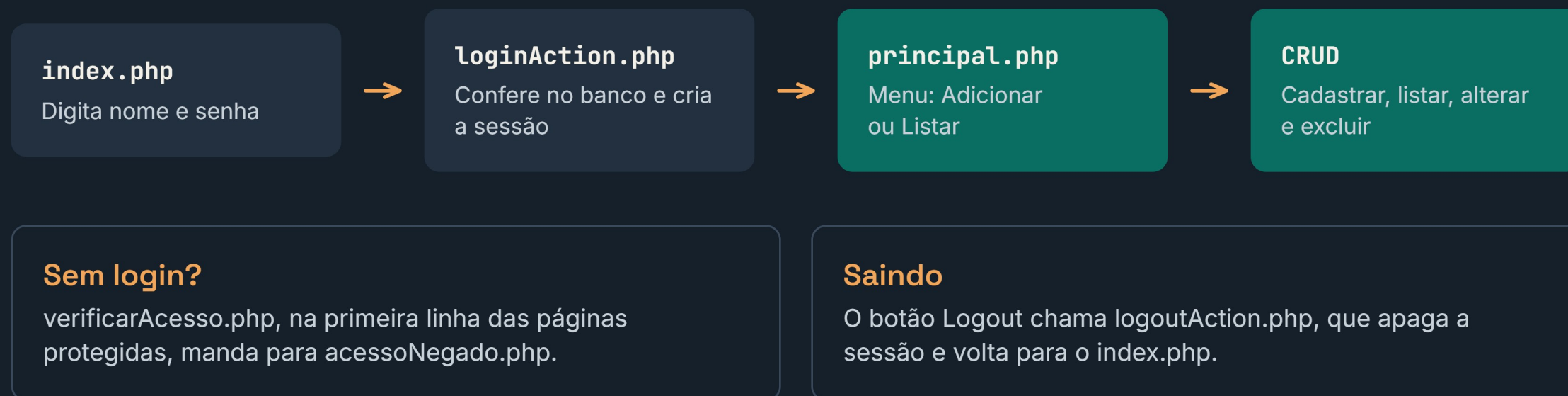
amigo

Campo	Tipo
idamigo	INT · PK · AI
nome	VARCHAR(45)
email	VARCHAR(45)
telefone	VARCHAR(45)

Guarda os dados que o CRUD cadastra, lista, altera e exclui.

FLUXO

Como o usuário navega pelo sistema



O formulário de login (index.php)

```
<form action="loginAction.php"
      method="post">
  <input type="text"
        name="txtNome" required>
  <input type="password"
        name="txtSenha" required>
  <button type="submit">
    Entrar
  </button>
</form>
```

action

Para onde os dados vão ao clicar em Entrar.

method="post"

Envia os dados escondidos, fora da URL.

name

A etiqueta que o PHP usa para achar o dado: \$ POST['txtNome'].

type="password"

Mostra a senha como bolinhas enquanto digita.

A conferência no banco (loginAction.php)

```
session_start();
$nome = $_POST['txtNome'];
$senha = $_POST['txtSenha'];
require_once 'conexaoBD.php';
$sql = "SELECT * FROM usuario WHERE nome = '". $nome . "'";
$resultado = $conexao->query($sql);
$linha = mysqli_fetch_array($resultado);
if ($linha != null) { // nome existe?
    if ($linha['senha'] == $senha) { // senha confere?
        $_SESSION['logado'] = $nome;
    } else { echo 'Login Inválido!'; }
} else { echo 'Login Inválido!'; }
$conexao->close();
```

1º if

O nome está na tabela usuario? Se não estiver, nem pergunta a senha.

2º if

A senha do banco é igual à senha digitada?

Deu certo

Cria \$_SESSION['logado'], a "pulseira" que libera as outras páginas.

REUTILIZAÇÃO

Escrever uma vez, usar em todas as páginas

conexaoBD.php

Servidor, usuário, senha e banco. Se a senha do banco mudar, altera só aqui.

cabecalho.php

DOCTYPE, head, W3.CSS e Font Awesome de todas as páginas.

rodape.php

Fecha o body e o html. Mudou aqui, mudou no site inteiro.

```
<?php require_once('verificarAcesso.php'); ?>
<?php require_once('cabecalho.php'); ?>
... conteúdo da página ...
<?php require_once('rodape.php'); ?>
```

Por que `require_once`? Se o arquivo faltar, dá Fatal Error e a página para, e ele nunca é importado duas vezes.

SEGURANÇA

Session: ninguém entra pela URL

```
// verificarAcesso.php
if (!isset($_SESSION)) {
    session_start();
}
if (!isset($_SESSION['logado'])) {
    header('location:acessoNegado.php');
    die();
}
```

O problema

O HTTP não guarda memória: sem proteção, digitar principal.php na URL pulava o login.

A solução

Toda página protegida chama verificarAcesso.php na primeira linha. Sem a sessão logado, header() redireciona e die() para o resto.

Cuidado

Ele não vai no index.php nem no loginAction.php, senão ninguém consegue fazer login.

CRUD

As quatro operações com os amigos

C

Cadastrar

```
INSERT INTO amigo  
(nome, email, ...)  
VALUES (...)
```

R

Listar

```
SELECT *  
FROM amigo
```

U

Alterar

```
UPDATE amigo  
SET nome = ...  
WHERE idamigo = ...
```

D

Excluir

```
DELETE FROM amigo  
WHERE idamigo = ...
```

Todas seguem o mesmo caminho: **formulário** → **arquivo de ação** → **comando SQL** → **mensagem com link de volta**.

READ

Listagem com foreach (listar.php)

```
require_once 'conexaoBD.php';
$sql = "SELECT * FROM amigo";
$resultado = $conexao->query($sql);
foreach ($resultado as $linha) {
    echo '<tr>';
    echo '<td>'.$linha['nome'].'</td>';
    echo '<td>'.$linha['email'].'</td>';
    echo '</tr>';
}
$conexao->close();
```

foreach

Percorre, uma por uma, todas as linhas que o banco devolveu.

\$linha['nome']

Pega o valor da coluna pelo nome dela.

echo

Cada volta do laço vira uma linha da tabela HTML.

LOGOUT

Saída segura (logoutAction.php)

```
<?php require_once('verificarAcesso.php'); ?>
<?php
    unset($_SESSION['logado']);
    header("location:index.php");
?>
```

unset()

Apaga a variável logado da sessão.

header()

Manda o usuário de volta para o login.

Resultado

Abriu principal.php de novo leva ao Acesso Negado.

PRÓXIMOS PASSOS

O que dá para melhorar na segurança

1

Senhas criptografadas

Hoje a senha fica em texto no banco. Com `password_hash()` e `password_verify()`, nem quem acessa o banco vê a senha.

2

Proteção contra SQL Injection

O SQL é montado com o texto digitado. Com prepared statements (mysqli ou PDO), o valor vai separado do comando.

3

Hospedagem com HTTPS

Publicar com domínio próprio e certificado SSL, para os dados viajarem criptografados.

HABILIDADES ADQUIRIDAS

O que este projeto me ensinou

Formulários HTML e \$_POST

Conexão com MySQL via mysqli

CRUD com INSERT, SELECT,
UPDATE e DELETE

Laços com foreach para
montar tabelas

Reutilização com require_once

Controle de acesso com session

Obrigada!

Perguntas?